Finance Tracker

Architecture du Frontend Angular

Documentation technique complète

26 mars 2026

Table des matières

1 Vue d'ensemble **3**
- 1.1 Stack technique . . . 3
- 1.2 Périmètre fonctionnel . . . 3

2 Structure des répertoires **3**

3 Configuration applicative (`app.config.ts`) **5**

4 Routage (`app.routes.ts`) **5**
- 4.1 Routes publiques . . . 5
- 4.2 Routes privées . . . 5

5 Couche Core – Infrastructure **6**
- 5.1 Client Supabase (`supabase.client.ts`) . . . 6
- 5.2 Service d'authentification (`auth.service.ts`) . . . 6
- 5.3 Garde d'authentification (`auth-guard.ts`) . . . 7
- 5.4 Garde invité (`guest-guard.ts`) . . . 7
- 5.5 Intercepteur HTTP (`auth.interceptor.ts`) . . . 8
- 5.6 Architecture du backend vue depuis le frontend . . . 8
- 5.7 Comment les éléments backend sont reliés entre eux . . . 9
- 5.8 Modèles partagés (`common.model.ts`) . . . 10
- 5.9 Validateur custom (`custom-validators.ts`) . . . 10

6 Couche Shared – Composants réutilisables **10**
- 6.1 PaginationComponent . . . 11

7 Couche Layout **11**

7.1 MainLayout . . . 11
7.2 AppSidebar (`features/sidebar/`) . . . 11

8 Couche Features – Services et modèles par domaine 12
8.1 Transactions . . . 12
8.1.1 Modèles . . . 12
8.1.2 Service . . . 12
8.1.3 Page . . . 13
8.2 Catégories . . . 13
8.3 Budgets . . . 13
8.4 Objectifs (`goals`) . . . 13
8.5 Dettes . . . 14
8.6 Transactions planifiées . . . 14
8.7 Calendrier . . . 15
8.8 Tableau de bord . . . 15
8.9 Import / Export CSV . . . 16
8.10 Profil . . . 16

9 Flux de données – exemple complet 17

10 Flux d'authentification 18
10.1 Connexion . . . 18
10.2 Accès à une route protégée . . . 18
10.3 Déconnexion . . . 18

11 Formulaires Reactive Forms 18

12 Référence des endpoints HTTP 19

13 Décisions d'architecture 21

14 Sécurité 21

15 Environnements 21

16 Conclusion 22

1 Vue d'ensemble

Finance Tracker est une application de gestion de finances personnelles. Le **frontend** est une Single Page Application (SPA) construite avec **Angular 21**, qui consomme une API REST exposée par un backend **Spring Boot** et délègue l'authentification à **Supabase Auth**.

1.1 Stack technique

Technologie	Rôle
Angular 21	Framework SPA, standalone components, lazy loading
TypeScript strict	Typage statique complet, aucun `any` autorisé
Angular Router	Navigation côté client, guards, lazy routes
Angular HttpClient	Requêtes HTTP, intercepteurs
Angular Reactive Forms	Formulaires typés avec validation
RxJS	Programmation réactive (Observables)
Supabase JS SDK	Authentification côté client
Tailwind CSS 4	Utilitaires CSS
chart.js + ng2-charts	Graphique linéaire du tableau de bord
SweetAlert2	Boîtes de dialogue et alertes
dayjs	Manipulation des dates

1.2 Périmètre fonctionnel

L'application couvre sept domaines métier :

- **Transactions** – saisie, filtrage, recherche, pagination
- **Catégories** – gestion des catégories revenus/dépenses
- **Budgets** – plafonds mensuels par catégorie avec alertes
- **Objectifs** – épargne progressive avec historique des contributions
- **Dettes** – suivi du remboursement avec historique des paiements
- **Transactions planifiées** – récurrences automatisées
- **Calendrier** – vue mensuelle avec projection des récurrences
- **Import / Export** – CSV bidirectionnel
- **Tableau de bord** – agrégats et visualisations

2 Structure des répertoires

Le projet suit une architecture en quatre couches : `core`, `shared`, `layout`, `features`.

```
src/app/
  app.ts                          # Composant racine
  app.config.ts                   # Configuration applicative (
     providers)
```

```
app.routes.ts                    # Définition des routes

core/                            # Infrastructure transverse
  auth/
    supabase.client.ts           # Client Supabase (singleton)
  guards/
    auth-guard.ts                # Protège les routes privées
    guest-guard.ts               # Bloque un user connecté sur /
       login
  interceptors/
    auth.interceptor.ts          # Injecte le Bearer token
  models/
    common.model.ts              # Types partagés (Page<T>,
       ApiError, enums)
  services/
    auth.service.ts              # Gestion de la session Supabase
    account.service.ts           # Déconnexion globale (logout-
       all)
  validators/
    custom-validators.ts         # passwordMatchValidator (
       factory)

shared/                          # Composants UI réutilisables
  components/
    pagination/                  # PaginationComponent (Spring
       Page)

layout/                          # Conteneurs visuels
  main-layout/                   # Sidebar + router-outlet

features/                        # Domaines métier
  landing/                       # Page d'accueil publique
  auth/                          # login, signup, forgot-password
     , reset-password
  dashboard/                     # Tableau de bord
  transactions/                  # CRUD transactions
  categories/                    # CRUD catégories
  budgets/                       # CRUD budgets
  goals/                         # Objectifs + contributions
  debts/                         # Dettes + paiements
  planned-transactions/          # Récurrences + runs
  calendar/                      # Calendrier mensuel
  import-export/                 # Import/export CSV
  profile/                       # Profil utilisateur
  sidebar/                       # Composant navigation (
     AppSidebar)
```

3 Configuration applicative (`app.config.ts`)

Angular 21 n'utilise plus de `NgModule` racine. La configuration est déclarative dans `app.config.ts` :

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideCharts(withDefaultRegisterables()),
  ]
};
```

— `provideZoneChangeDetection` – active la détection de changements avec coalescing (une seule passe par macro-tâche au lieu d'une par événement).

— `provideRouter(routes)` – enregistre les routes avec lazy loading.

— `provideHttpClient(withInterceptors([...]))` – configure `HttpClient` en mode standalone et branche `authInterceptor` sur **toutes** les requêtes HTTP sortantes.

— `provideCharts` – enregistre les composants chart.js nécessaires à `ng2-charts`.

— Tous les services métier utilisent `providedIn: 'root'` : ils sont des **singletons** créés à la demande par l'injecteur Angular.

4 Routage (`app.routes.ts`)

4.1 Routes publiques

URL	Composant	Guard
`/`	`LandingPage`	–
`/login`	`LoginPage`	`guestGuard`
`/signup`	`SignupPage`	`guestGuard`
`/forgot-password`	`ForgotPasswordPage`	`guestGuard`
`/reset-password`	`ResetPasswordPage`	–

4.2 Routes privées

Toutes sous `/app`, encapsulées dans `MainLayout` et protégées par `authGuard` :

URL	Composant
/app/dashboard	DashboardPage
/app/transactions	TransactionsPage
/app/categories	CategoriesPage
/app/budgets	BudgetsPage
/app/goals	GoalsPage
/app/debts	DebtsPage
/app/planned	PlannedPage
/app/calendar	CalendarPage
/app/import-export	ImportExportPage
/app/profile	ProfilePage

Chaque route utilise `loadComponent()` : le bundle JavaScript du composant est téléchargé **uniquement** lorsque l'utilisateur accède à la page (*lazy loading*).

```
{
  path: 'transactions',
  loadComponent: () =>
    import('./features/transactions/pages/transactions-page/
       transactions-page')
      .then(m => m.TransactionsPage)
}
```

5 Couche Core – Infrastructure

5.1 Client Supabase (`supabase.client.ts`)

Le client Supabase est instancié **une seule fois** au démarrage de l'app, avec la clé publique `anon` (jamais la `service_role`) :

```
export const supabase = createClient(
  environment.supabaseUrl,
  environment.supabaseAnonKey,
  {
    auth: {
      lock: (_name, _timeout, fn) => fn(),
    },
  }
);
```

L'option `lock` désactive le verrou interne de Supabase, qui utilise normalement la Web Locks API (`navigator.locks`). Cela permet à `AuthService` de lire la session de façon **synchrone** depuis le cache, sans attendre une Promise à chaque requête HTTP.

5.2 Service d'authentification (`auth.service.ts`)

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private session$ = new BehaviorSubject<Session | null>(null);
  private initialized = false;

  constructor(private zone: NgZone) {
```

```
    supabase.auth.getSession().then(({ data }) => {
      this.session$.next(data.session);
      this.initialized = true;
    });
    supabase.auth.onAuthStateChange((_event, session) => {
      this.zone.run(() => this.session$.next(session));
    });
  }

  getAccessToken(): string | null {
    return this.session$.value?.access_token ?? null;
  }

  isAuthenticated(): boolean {
    return this.session$.value !== null;
  }
}
```

Mécanismes clés :

- `BehaviorSubject<Session | null>` – émet la session courante à chaque changement ; valeur initiale `null` (non connecté).
- `waitForInit()` – les guards attendent que `getSession()` ait répondu avant de décider si l'utilisateur est authentifié.
- `getAccessToken()` – lecture **synchrone** du token depuis `BehaviorSubject.value`. Supabase met à jour le cache localement ; l'intercepteur n'a pas besoin de souscrire à un Observable.
- `NgZone.run()` – les événements Supabase arrivent hors de la zone Angular ; cette ligne force la détection de changements lors du changement de session.

5.3 Garde d'authentification (`auth-guard.ts`)

```
export const authGuard: CanActivateFn = async () => {
  const auth   = inject(AuthService);
  const router = inject(Router);
  await auth.waitForInit();
  if (!auth.isAuthenticated())
    return router.createUrlTree(['/login']);
  return true;
};
```

Appliqué au groupe de routes `/app/**`. Si la session n'est pas encore chargée au démarrage (premier chargement de l'app), le guard **attend** la résolution de `waitForInit()` avant de statuer.

5.4 Garde invité (`guest-guard.ts`)

Empêche un utilisateur **déjà connecté** d'accéder à `/login` ou `/signup`. Il est redirigé vers `/app/dashboard`.

```
export const guestGuard: CanActivateFn = async () => {
  const auth   = inject(AuthService);
  const router = inject(Router);
  await auth.waitForInit();
```

```
  if (auth.isAuthenticated())
    return router.createUrlTree(['/app/dashboard']);
  return true;
};
```

5.5 Intercepteur HTTP (`auth.interceptor.ts`)

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getAccessToken();
  if (!token) return next(req);
  return next(req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  }));
};
```

Fonctionnement :

1. Chaque requête HTTP passe par cet intercepteur (enregistré globalement).
2. Le token JWT est lu **synchronement** depuis le cache d'AuthService.
3. Si un token existe, la requête est clonée (immuabilité) avec l'en-tête `Authorization: Bearer <token>`.
4. Si aucun token n'existe (utilisateur non connecté), la requête passe telle quelle.
5. Le backend Spring Boot valide ce JWT sur toutes les routes protégées.

5.6 Architecture du backend vue depuis le frontend

Même si ce dépôt contient uniquement le frontend Angular, le contrat HTTP et les types manipulés permettent de reconstituer précisément la **chaîne de traitement** côté backend. Le modèle le plus cohérent avec les endpoints utilisés est une architecture Spring Boot classique en couches :

Couche backend	Responsabilité
Security filter chain	Intercepte la requête entrante, lit l'en-tête `Authorization`, valide le JWT Supabase, refuse la requête si le token est absent ou invalide sur une route protégée.
Controller REST	Expose les routes `/v1/**`, reçoit le JSON, valide les données entrantes, convertit la requête HTTP en appel métier.
Service métier	Porte les règles métier : création, recherche, contrôle des droits, calculs, déclenchement d'actions liées, orchestration entre plusieurs repositories si nécessaire.
Repository / persistance	Charge et sauvegarde les entités en base, applique les filtres, tris, paginations et recherches demandés par le service.
DTO / sérialisation	Transforme les objets métier en JSON de réponse stable pour le frontend : listes, pages, messages d'erreur, statuts, enums.

Chaîne complète d'une requête protégée :

1. Le frontend appelle un endpoint via `HttpClient`.

2. L'intercepteur Angular ajoute le `Bearer token`.
3. Spring Security intercepte la requête avant le contrôleur.
4. Le JWT est vérifié ; l'identité utilisateur est extraite du token.
5. Le contrôleur appelle le service du domaine concerné.
6. Le service applique les règles métier et utilise un ou plusieurs repositories.
7. Les données sont relues ou persistées en base.
8. Le backend construit la réponse JSON finale et renvoie le code HTTP adapté (`200`, `201`, `204`, `400`, `401`, etc.).

Point important : le frontend **n'envoie jamais l'identité utilisateur** dans ses payloads métier. Le backend relie chaque opération au bon utilisateur à partir du JWT validé. Cette décision simplifie le contrat, réduit le risque d'usurpation et garantit que les données lues ou modifiées appartiennent bien à la session authentifiée.

5.7 Comment les éléments backend sont reliés entre eux

Les routes visibles dans le frontend montrent une séparation nette par domaine. Chaque domaine est très probablement raccordé à un ensemble cohérent de classes backend reliées entre elles :

— **Transactions** : un contrôleur reçoit les CRUD et recherches, un service applique les règles de création/mise à jour, puis un repository persiste les lignes de transaction.

— **Catégories** : les transactions référencent une catégorie ; le backend doit donc garantir qu'une catégorie utilisée appartient au même utilisateur et reste cohérente avec le type revenu/dépense.

— **Budgets** : les budgets sont liés aux catégories et à une période. Le backend agrège ou compare les transactions du mois avec les plafonds.

— **Objectifs** et **dettes** : chaque entité principale possède un historique paginé (`GoalContribution`, `DebtPayment`), ce qui implique un lien parent-enfants côté persistance et des services capables de recalculer les soldes après chaque mouvement.

— **Transactions planifiées** : elles produisent des exécutions (`PlannedTransaction Run`) et sont connectées au calendrier ainsi qu'au flux transactionnel réel lorsqu'une occurrence est lancée.

— **Calendrier** : il agrège les événements mensuels visibles dans l'UI ; il dépend donc soit de ses propres données, soit de projections calculées à partir des transactions planifiées.

— **Compte / sécurité** : `/account/logout-all` ne manipule pas les données métier ; il s'appuie sur l'identité extraite du token pour invalider les sessions côté serveur.

En pratique, la liaison entre ces éléments suit presque toujours le même axe : **Controller → Service → Repository → base de données**, puis la réponse remonte en sens inverse. Le frontend ne connaît que le **contrat HTTP** ; tout le reste est encapsulé dans le backend, ce qui permet de faire évoluer la logique métier sans casser l'interface Angular tant que les DTO restent compatibles.

5.8 Modèles partagés (`common.model.ts`)

```typescript
// Types union -- correspondent aux enums backend
export type TransactionType   = 'INCOME' | 'EXPENSE';
export type PlannedFrequency  = 'DAILY' | 'WEEKLY' | 'MONTHLY' | 'YEARLY';
export type GoalStatus        = 'ACTIVE' | 'COMPLETED' | 'ARCHIVED';
export type DebtStatus        = 'ACTIVE' | 'PAID' | 'CANCELLED';
export type PlannedRunStatus  = 'SUCCESS' | 'FAILED';

// Modèle d'erreur standardisé retourné par le backend
export interface ApiError {
  timestamp: string;
  status: number;
  error: string;
  message: string;
  path: string;
  details?: string[];
}

// Pagination Spring (page 0-indexée)
export interface Page<T> {
  content: T[];
  totalElements: number;
  totalPages: number;
  number: number;  // page courante (0-based)
  size: number;
}
```

5.9 Validateur custom (`custom-validators.ts`)

Factory de validateur Angular utilisée sur les formulaires de changement de mot de passe :

```typescript
export function passwordMatchValidator(
  passwordField = 'password',
  confirmField  = 'confirmPassword',
): ValidatorFn {
  return (ctrl: AbstractControl): ValidationErrors | null => {
    const pw  = ctrl.get(passwordField)?.value;
    const cpw = ctrl.get(confirmField)?.value;
    return pw && cpw && pw !== cpw ? { passwordMismatch: true } : null;
  };
}
```

C'est une **factory** (fonction qui retourne un `ValidatorFn`) afin de configurer les noms des champs à comparer selon le formulaire :

- `signup-page` et `reset-password-page` : champs `password` + `confirmPassword` (valeurs par défaut)
- `profile-page` : champs `newPassword` + `confirmPassword`

6 Couche Shared – Composants réutilisables

6.1 PaginationComponent

Composant générique de pagination qui consomme le format `Page<T>` de Spring :

```
@Component({ selector: 'app-pagination', standalone: true, ... })
export class PaginationComponent {
  @Input() page          = 0;          // 0-based (Spring)
  @Input() totalPages    = 0;
  @Input() totalElements = 0;
  @Input() size          = 20;
  @Output() pageChange   = new EventEmitter<number>();
}
```

Comportement :

- Affiche « N–M sur TOTAL » (converti en 1-based pour l'utilisateur).
- Génère une liste intelligente de boutons de page : toujours les 2 premières et les 2 dernières pages, plus une fenêtre de 2 pages autour de la page courante.
- Insère des points de suspension (...) lorsqu'il y a un saut.
- Boutons Précédent / Suivant désactivés aux extrémités.
- Émet l'événement `pageChange` sur sélection d'une page.

7 Couche Layout

7.1 MainLayout

Conteneur de toutes les pages privées (`/app/**`) :

```
@Component({
  selector: 'app-main-layout',
  standalone: true,
  imports: [RouterOutlet, AppSidebar],
})
export class MainLayout {}
```

Le template positionne la sidebar à gauche (largeur fixe 256 px) et le contenu principal à droite :

```
<div class="flex min-h-screen bg-[#f6f6f8]">
  <app-sidebar />
  <div class="ml-64 flex-1">
    <router-outlet />
  </div>
</div>
```

7.2 AppSidebar (`features/sidebar/`)

Composant de navigation latérale chargé par `MainLayout`.

Structure de la navigation (sections) :

- *Vue d'ensemble* : Tableau de bord, Transactions, Planifiées
- *Gestion* : Catégories, Budgets, Objectifs, Dettes

— *Outils* : Calendrier, Import / Export

— *Compte* : Paramètres (profile)

Fonctionnalités :

— Affiche le nom complet et les initiales de l'utilisateur (lues depuis les métadonnées Supabase).

— Menu déroulant utilisateur fermé par `HostListener('document:click')`.

— Déconnexion locale via SweetAlert2 → `authService.signOut()` → redirection vers `/`.

8 Couche Features – Services et modèles par domaine

Chaque domaine métier suit la même organisation :

```
features/<domaine>/
  models/<domaine>.model.ts     # Interfaces TypeScript
  services/<domaine>.service.ts # Appels HTTP
  pages/<domaine>-page/
    <domaine>-page.ts           # Logique du composant
    <domaine>-page.html         # Template
    <domaine>-page.css/scss
```

8.1 Transactions

8.1.1 Modèles

```
export interface Transaction {
  id: number;          label: string;       amount: number;
  type: TransactionType;  categoryId: number; date: string; // YYYY-
      MM-DD
  note?: string | null;   source?: string | null;
  createdAt: string;      updatedAt?: string | null;
}

export interface CreateTransactionRequest {
  amount: number;   type: TransactionType;  categoryId: number;
  label: string;    date: string;           note?: string | null;
}

export interface TransactionSearchParams {
  type?: TransactionType;  categoryId?: number;
  startDate?: string;      endDate?: string;    label?: string;
}
```

8.1.2 Service

```
@Injectable({ providedIn: 'root' })
export class TransactionsService {
  private base = `${environment.apiBaseUrl}/transactions`;

```

```
  getAll(page = 0, size = 200): Observable<Page<Transaction>>
  search(params: TransactionSearchParams, page = 0, size = 200):
      Observable<Page<Transaction>>
  create(payload: CreateTransactionRequest): Observable<Transaction>
  update(id: number, payload: UpdateTransactionRequest): Observable<
     Transaction>
  delete(id: number): Observable<void>
}
```

8.1.3 Page

Charge **toutes** les transactions en mémoire (`size=200`), puis applique le filtrage et la pagination **côté client**. Ce choix simplifie l'implémentation pour un projet académique à faible volumétrie.

8.2 Catégories

```
export interface Category {
  id: number;   name: string;
  icon?: string | null;   color?: string | null;
  isDefault: boolean;     createdAt: string;
}
```

La page distingue deux onglets (*Dépenses* / *Revenus*) et propose :

— Une banque de 60+ icônes Material Symbols avec recherche textuelle.

— 12 couleurs prédéfinies pour l'icône.

— La suppression est bloquée sur les catégories `isDefault = true`.

8.3 Budgets

```
export interface Budget {
  id: number;           categoryId: number;
  monthlyLimit: number; alertThreshold: number; // 0-100
  createdAt: string;    updatedAt?: string | null;
}
```

La page enrichit chaque budget avec le nom et la couleur de la catégorie associée. Le seuil d'alerte (`alertThreshold`) est un pourcentage : si le montant dépensé atteint ce seuil, le budget s'affiche en orange ou rouge.

8.4 Objectifs (`goals`)

```
export interface Goal {
  id: number;           name: string;
  targetAmount: number; currentAmount: number;
  deadline?: string | null;  status: GoalStatus;
}

export interface GoalContribution {
  id: number;           goalId: number;
  amount: number;       contributionDate: string;
  note?: string | null; transactionId?: number | null;
```

```
}

export interface ContributeGoalRequest {
  amount: number;  contributionDate: string;  note?: string | null;
}
```

Endpoints spéciaux :

— `PATCH /v1/goals/{id}/contribute` – ajoute une contribution.

— `GET /v1/goals/{id}/contributions` – historique paginé.

La page affiche des cercles de progression via un `conic-gradient` CSS calculé dynamiquement. Les contributions sont chargées **à la demande** (*lazy*) par objectif.

8.5 Dettes

```
export interface Debt {
  id: number;              creditorName: string;
  totalAmount: number;     remainingAmount: number;
  monthlyPayment: number;  interestRate?: number | null;
  startDate: string;       dueDate?: string | null;
  status: DebtStatus;
}

export interface DebtSummary {
  activeDebtsCount: number;   paidDebtsCount: number;
  totalRemainingAmount: number; totalMonthlyPayment: number;
}
```

Endpoints spéciaux :

— `PATCH /v1/debts/{id}/payment` – enregistre un paiement.

— `GET /v1/debts/{id}/payments` – historique paginé.

— `GET /v1/debts/summary` – agrégats globaux.

8.6 Transactions planifiées

```
export interface PlannedTransaction {
  id: number;          label: string;     amount: number;
  type: TransactionType; categoryId: number;
  frequency: PlannedFrequency;  nextDueDate: string;
  autoCreate: boolean; active: boolean;
}
```

Endpoints spéciaux :

— `PATCH /v1/planned-transactions/{id}/toggle` – active/désactive.

— `POST /v1/planned-transactions/{id}/run-now` – exécution manuelle.

— `GET /v1/planned-transactions/{id}/runs` – historique d'exécutions.

La page applique plusieurs règles UI spécifiques :

— La carte « Prochaine échéance » trie les planifications filtrées par `nextDueDate` croissant et affiche la plus proche échéance, au lieu de reprendre le premier élément brut renvoyé par l'API.

— Le formulaire de création/édition force les champs `amount`, `type`, `frequency`, `nextDueDate` et `autoCreate` avec des contrôles `nonNullable`, pour ne jamais envoyer `null` sur ces valeurs au backend.

— Après une exécution manuelle réussie (`run-now`), la page recharge la liste complète afin de récupérer le `nextDueDate` recalculé côté backend et de rafraîchir immédiatement la carte ainsi que le tableau.

8.7 Calendrier

```
export interface CalendarEvent {
  id: number;   title: string;   eventDate: string; // YYYY-MM-DD
  amount?: number | null;   color?: string | null;
  notes?: string | null;
}
```

La page construit une grille de 42 jours (6 semaines Œ 7). Les transactions planifiées actives sont **projetées** dans la grille selon leur fréquence :

— `DAILY` – apparaît chaque jour.

— `WEEKLY` – même jour de semaine toutes les 7 jours.

— `MONTHLY` – même date chaque mois.

— `YEARLY` – même date et même mois.

8.8 Tableau de bord

```
export interface MonthlySummary {
  month: string; totalIncome: number;
  totalExpenses: number; netBalance: number;
  transactionCount: number;
}

export interface SpendingByCategory {
  categoryId: number; categoryName: string;
  amount: number; percentage: number;
}

export interface GoalProgress {
  id: number; name: string;
  targetAmount: number; currentAmount: number;
  percentage: number; status: GoalStatus;
}

export interface BudgetAlert {
  categoryName: string; monthlyLimit?: number;
  spentAmount?: number; percentage?: number;
  overBudget?: boolean;
}
```

La page lance un `forkJoin` qui déclenche **7 requêtes HTTP en parallèle** au chargement :

```
forkJoin({
  summary:   dashboardService.getSummary(month),
  spending:  dashboardService.getSpendingByCategory(month),
```

```
  planned:   plannedService.getAll(),
  goals:     dashboardService.getGoalsProgress(),
  alerts:    dashboardService.getBudgetAlerts(month),
  recent:    transactionsService.getAll(0, 5),
  monthlyTx: transactionsService.search({ startDate, endDate }, 0,
      500),
}).subscribe({ next: (data) => { /* populate widgets */ } });
```

Chaque requête est protégée par `catchError(() => of(valeurParDefaut))` afin qu'une panne partielle ne bloque pas l'ensemble du tableau de bord.

Le seul graphique chart.js de l'application (ligne chronologique) est ici :

```
lineData: ChartData<'line'> = { labels: [], datasets: [{
  data: [], borderColor: '#0f766e', fill: true, tension: 0.4
}]};
// Dans le template :
// <canvas baseChart [data]="lineData" [options]="lineOptions" type="
    line">
```

Le graphique "donut" de répartition des dépenses est du **CSS pur** :

```
get spendingBackground(): string {
  // Construit un conic-gradient avec les proportions des catégories
  return `conic-gradient(${stops.join(',')})`;
}
```

8.9 Import / Export CSV

Export : récupère toutes les transactions via `getAll(0,10000)`, génère un `Blob` CSV avec BOM UTF-8 (pour compatibilité Excel) et déclenche un téléchargement via un lien `<a>` créé dynamiquement.

Import : lit le fichier CSV via `FileReader`, valide chaque ligne, puis appelle `create()` pour chaque ligne valide. Les erreurs de parsing et les erreurs HTTP sont collectées et affichées dans un rapport détaillé.

Format CSV attendu :

```
date,label,amount,type,categoryId,note
2025-03-01,Courses,45.50,EXPENSE,3,Supermarché
2025-03-02,Salaire,2500,INCOME,1,
```

8.10 Profil

Le service `ProfileService` appelle directement le **SDK Supabase** (pas le backend Spring) pour mettre à jour le nom et le mot de passe :

```
updateProfile(req: ProfileUpdateRequest): Promise<User> {
  return supabase.auth.updateUser({
    data: { full_name: req.fullName }
  });
}

updatePassword(req: PasswordUpdateRequest): Promise<User> {
```

```
  return supabase.auth.updateUser({ password: req.newPassword });
}
```

La page `ProfilePage` inclut également la déconnexion globale via `AccountService` (situé dans `core/services/`), qui appelle `POST /v1/account/logout-all` sur le backend Spring pour invalider toutes les sessions serveur.

9 Flux de données – exemple complet

Exemple : création d'une transaction

1. L'utilisateur remplit le formulaire Reactive Forms et clique sur « Créer ».
2. Le composant vérifie `form.invalid` : si invalide, tous les champs sont marqués `touched` pour afficher les erreurs ; on retourne.
3. `mapFormToRequest()` construit le payload : un `CreateTransactionRequest` typé.
4. `transactionsService.create(payload)` est appelé. Il retourne un `Observable<Transaction>`.
5. La requête `POST /v1/transactions` est construite par `HttpClient`.
6. `authInterceptor` lit le JWT depuis le cache et ajoute `Authorization: Bearer <token>`.
7. Le filtre de sécurité Spring valide le JWT et rattache la requête à l'utilisateur authentifié.
8. Le contrôleur REST reçoit le JSON, le service métier vérifie les règles de création, puis le repository persiste la transaction en base.
9. Le backend retourne `201 Created` avec le `Transaction` JSON sérialisé pour le frontend.
10. `HttpClient` désérialise en `Transaction`.
11. Callback `next` : fermeture du formulaire, rechargement de la liste, toast SweetAlert2 succès (`timer: 1500`).
12. Callback `error` : toast SweetAlert2 erreur avec message explicite.

Exemple : exécution manuelle d'une transaction planifiée

1. L'utilisateur clique sur l'action `run-now` depuis une ligne de la page `PlannedPage`.
2. Une confirmation SweetAlert2 sécurise l'action avant l'appel HTTP.
3. `plannedService.runNow(id)` envoie `POST /v1/planned-transactions/{id}/run-now`.
4. Le backend crée la transaction réelle, enregistre une ligne `PlannedTransactionRun`, puis avance `nextDueDate` selon la fréquence de la planification.
5. En cas de succès, le frontend recharge les planifications et, si le détail est ouvert, recharge aussi l'historique des exécutions.
6. La colonne `nextDueDate` du tableau et la carte « Prochaine échéance » affichent alors immédiatement la nouvelle date renvoyée par le backend.

10 Flux d'authentification

10.1 Connexion

1. L'utilisateur soumet le formulaire `LoginPage`.
2. `authService.signIn(email, password)` appelle le SDK Supabase.
3. Supabase retourne un `Session` (contenant le JWT).
4. `BehaviorSubject` est mis à jour via l'événement `onAuthStateChange`.
5. Le router navigue vers `/app/dashboard`.
6. Désormais, chaque requête HTTP inclura automatiquement le token via l'intercepteur.

10.2 Accès à une route protégée

1. L'utilisateur accède à `/app/transactions`.
2. `authGuard` s'exécute : `await auth.waitForInit()`.
3. Si session absente → redirection vers `/login`.
4. Si session présente → la route est activée, `MainLayout` est rendu.

10.3 Déconnexion

1. L'utilisateur clique sur « Se déconnecter » dans la sidebar.
2. SweetAlert2 demande confirmation.
3. `authService.signOut()` vide la session Supabase locale.
4. `BehaviorSubject` émet `null`.
5. Le router redirige vers `/`.
6. L'intercepteur ne trouvant plus de token, les prochaines requêtes seront anonymes.

11 Formulaires Reactive Forms

Tous les formulaires métier utilisent **Angular Reactive Forms**. Le pattern appliqué dans chaque composant est :

```
// Déclaration du groupe
form = this.fb.group({
  label:      this.fb.nonNullable.control('', [Validators.required,
     Validators.maxLength(80)]),
  amount:     this.fb.nonNullable.control(0, [Validators.required,
     Validators.min(0.01)]),
  type:       this.fb.nonNullable.control('EXPENSE' as
     TransactionType, [Validators.required]),
  categoryId: [null as number | null, [Validators.required]],
  frequency:  this.fb.nonNullable.control('MONTHLY' as
     PlannedFrequency, [Validators.required]),
  nextDueDate:this.fb.nonNullable.control(dayjs().format('YYYY-MM-DD'
     ), [Validators.required]),
  autoCreate: this.fb.nonNullable.control(false),
```

```
});

// Accesseurs typés (évite les .get('label') répétés dans le template)
get label()       { return this.form.controls.label; }
get amount()      { return this.form.controls.amount; }

// Soumission
submit(): void {
  if (this.form.invalid) { this.form.markAllAsTouched(); return; }
  const payload = this.mapFormToRequest();
  this.service.create(payload).subscribe({ ... });
}
```

Cette approche réduit les erreurs de sérialisation côté backend : les champs primitifs ou enum ne partent plus avec `null` dans le JSON, ce qui est particulièrement important pour des propriétés comme `boolean autoCreate`.

Le template exploite les états Angular pour l'UX :

```
<input formControlName="label"
       [class.border-red-500]="label.invalid && label.touched"
       required maxlength="80" />
@if (label.invalid && label.touched) {
  <p class="text-red-500 text-sm mt-1">
    @if (label.hasError('required'))  { Le libellé est requis. }
    @if (label.hasError('maxlength')) { 80 caractères maximum. }
  </p>
}
<button [disabled]="form.invalid || saving" type="submit">
  Créer
</button>
```

12 Référence des endpoints HTTP

Base URL : `http://localhost:8080/v1`

Méthode	Endpoint	Retour
GET	/transactions	Page<Transaction>
GET	/transactions/search	Page<Transaction>
POST	/transactions	Transaction
PUT	/transactions/{id}	Transaction
DELETE	/transactions/{id}	void
GET	/categories	Category[]
POST	/categories	Category
PUT	/categories/{id}	Category
DELETE	/categories/{id}	void
GET	/budgets	Budget[]
POST	/budgets	Budget
PUT	/budgets/{id}	Budget

DELETE	/budgets/{id}	void
GET	/goals	Goal[]
POST	/goals	Goal
PUT	/goals/{id}	Goal
DELETE	/goals/{id}	void
PATCH	/goals/{id}/contribute	Goal
GET	/goals/{id}/contributions	Page<GoalContribution>
GET	/debts	Debt[]
POST	/debts	Debt
PUT	/debts/{id}	Debt
DELETE	/debts/{id}	void
PATCH	/debts/{id}/payment	Debt
GET	/debts/{id}/payments	Page<DebtPayment>
GET	/debts/summary	DebtSummary
GET	/planned-transactions	PlannedTransaction[]
POST	/planned-transactions	PlannedTransaction
PUT	/planned-transactions/{id}	PlannedTransaction
DELETE	/planned-transactions/{id}	void
PATCH	/planned-transactions/{id}/toggle	PlannedTransaction
POST	/planned-transactions/{id}/run-now	PlannedTransactionRun
GET	/planned-transactions/{id}/runs	Page< PlannedTransactionRun>
GET	/calendar/events?month=YYYY-MM	CalendarEvent[]
POST	/calendar/events	CalendarEvent
PUT	/calendar/events/{id}	CalendarEvent
DELETE	/calendar/events/{id}	void
GET	/dashboard/summary?month=YYYY-MM	MonthlySummary
GET	/dashboard/spending-by-category	SpendingByCategory[]
GET	/dashboard/goals-progress	GoalProgress[]
GET	/dashboard/budget-alerts	BudgetAlert[]
POST	/account/logout-all	void

13 Décisions d'architecture

Décision	Justification
Standalone components	Plus de NgModule ; chaque composant déclare ses propres imports. Plus lisible, plus modulaire, conforme aux bonnes pratiques Angular 17+.
Lazy loading sur toutes les routes	Le bundle initial est minimal. Le code d'une page n'est téléchargé que lorsque l'utilisateur y accède.
BehaviorSubject pour la session	Lecture synchrone du token dans l'intercepteur. Pas besoin de souscrire à un Observable à chaque requête.
Un service par domaine	Séparation stricte des responsabilités. Chaque service connaît uniquement ses propres endpoints.
Filtrage côté client (transactions)	Simplification pour une volumétrie faible. Les endpoints de recherche backend existent pour un usage en production.
Pas d'intercepteur d'erreur centralisé	Chaque page gère ses erreurs avec le contexte métier approprié (message adapté à l'action).
Reactive Forms uniquement	Validation complexe (cross-fields, conditions) impossible proprement en template-driven forms.
SweetAlert2	UX supérieure à `window.alert/confirm`. Couleurs cohérentes avec la charte graphique.
Pas d'envoi de `userId`	Le backend identifie l'utilisateur depuis le JWT. Aucun identifiant ne transite dans le corps des requêtes.

14 Sécurité

— La clé Supabase `anon` est la seule clé présente côté frontend. Elle est publique par définition et ne donne accès qu'aux ressources que les règles RLS (Row Level Security) Supabase autorisent.

— La clé `service_role` (clé admin Supabase) n'est **jamais** exposée côté frontend.

— Le JWT n'est jamais loggé ni stocké en dehors du cache interne du SDK Supabase (localStorage ou cookies selon la configuration Supabase).

— Le backend Spring valide le JWT sur chaque requête protégée et extrait l'identité de l'utilisateur depuis le token, jamais depuis le corps de la requête.

— `logout-all` appelle `POST /v1/account/logout-all` sans paramètre `userId` : le backend déduit l'utilisateur depuis le token.

15 Environnements

```
1 // environment.ts (développement)
2 export const environment = {
3   production:      false,
4   apiBaseUrl:      'http://localhost:8080/v1',
5   supabaseUrl:     'https://YOUR_PROJECT.supabase.co',
```

```
  supabaseAnonKey: 'YOUR_SUPABASE_ANON_KEY',
};
```

Toutes les URL backend sont construites relativement à `environment.apiBaseUrl`. Il n'existe **qu'un seul préfixe** dans l'application : `/v1`.

16 Conclusion

Le frontend Finance Tracker est une SPA Angular bien structurée, strictement typée et sécurisée. Les principes de conception retenus – séparation claire des couches, lazy loading, Reactive Forms, singleton services, token JWT via intercepteur – en font une base maintenable et extensible.

Chaque domaine métier est autonome : ses modèles, son service et ses composants sont colocalisés dans `features/<domaine>/`. Les parties transverses (auth, guards, intercepteurs, types communs) sont isolées dans `core/`. Les composants réutilisables vivent dans `shared/`.

Cette architecture est conçue pour rester lisible et explicable à mesure que le projet évolue, sans sur-ingénierie (pas de state management externe, pas de multi-comptes, pas de devise multiple).